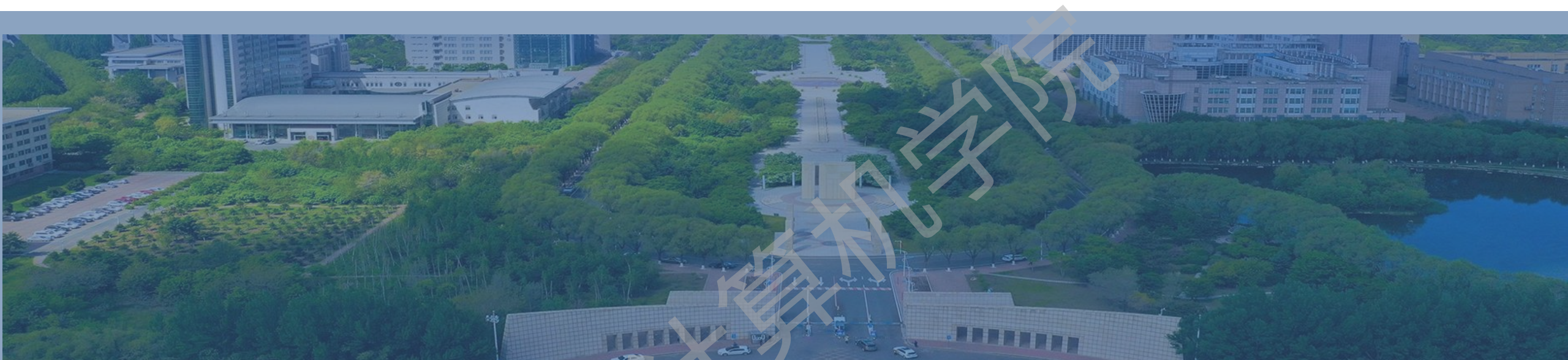




第五章 贪心方法





目录

- 5.1 一般方法
- 5.2 背包问题
- 5.3 带有期限的作业调度问题
- 5.4 最小生成树问题
- 5.5 货郎担问题
- 5.6 小结





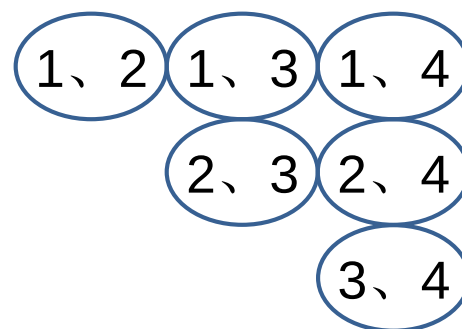
5.1 一般方法

- 方法适用的问题特点
- 方法的基础知识
- 方法的求解步骤及核心问题
- 方法的抽象化控制
- 方法的缺点和优点



一个现实世界中的例子

- 如果有一个机会可以帮助你梦想成真，那么你希望参加哪两个活动：
 - 1 看足球世界杯总决赛；
 - 2 游览新西兰风光；
 - 3 学习潜水；
 - 4 听一场演唱会。



贪心方法适用的问题特点

- 有这样一类问题：它有 n 个输入，而它的解就是这 n 个输入的某个子集，这些子集必须满足某些事先给定的条件。
 - 约束条件：必须满足的条件
 - 可行解：满足约束条件的子集
 - 目标函数：为了衡量可行解的优劣，预先给定衡量标准，以函数形式给出
 - 最优解：使目标函数取极值(极大值或极小值)的可行解

思考：上述术语和上一页的例子如何对应？



贪心法的求解思想

- 贪心方法是一种“只顾眼前”的分级处理方法：
 - 根据题意选取一种量度标准；
 - 按该标准一次选中一个输入；
 - 如果这个输入和当前的部分解加在一起满足约束条件，则将其加入到部分解中；否则舍弃掉。

思考：贪心法得到的可行解是否一定是问题的最优解？
取决于贪心法中的哪个环节？

量度标准



算法5.1 贪心法的抽象化控制

Procedure GREEDY(A, n)

//A(1:n)包含n个输入

solution $\leftarrow\Phi$ //解向量solution初始化为空

for i $\leftarrow$ 1 to n do

x $\leftarrow$ **SELECT**(A)

if **FEASIBLE**(solution,x)

then solution $\leftarrow$ **UNION**(solution,x)

endif

repeat

return (solution)

end GREEDY

- **SELECT**: 按某种最优量度标准从A中选择一个输入赋值给x,并从A中消去它
- **FEASIBLE**: 判定x是否可以包含在解向量中。
- **UNION**: 将x与解向量结合并修改约束判定。



贪心法的特点

- 贪心法设计求解的核心问题是
 - 选择能产生问题最优解的量度标准，即最优量度标准
- 缺点：
 - 不是对所有问题都能得到最优解
 - 基于目标函数制定的度量标准不一定是最优的
 - 最优量度标准需要经过证明
- 优点：
 - 一旦证明成立后，它就是一种高效的算法
 - 策略的构造简单易行
 - 对许多问题都能产生整体最优解或者近似最优解





5.2 背包问题

- 问题描述
- 问题实例
- 贪心算法
- 贪心解最优证明



问题描述

- 已知有 n 种物品和一个可容纳 M 重量的背包，每种物品 i 的重量为 w_i ，假定将物品 i 的某一部分 x_i 放入背包就会得到 $p_i x_i$ 的效益($0 \leq x_i \leq 1, p_i > 0$)，采用怎样的装包方法会使装入背包物品的总效益为最大？
- 问题的形式化描述：

极大化	$\sum_{1 \leq i \leq n} p_i x_i$	$0 \leq x_i \leq 1, p_i > 0$
约束条件	$\sum_{1 \leq i \leq n} w_i x_i \leq M$	$w_i > 0, 1 \leq i \leq n$



背包问题实例

- 考虑下列情况下的背包问题：

- $n=3, M=20, (p_1, p_2, p_3)=(25, 24, 15), (w_1, w_2, w_3)=(18, 15, 10)$

- 分析：

- 按效益值的非增次序

(x_1, x_2, x_3)

$(1, 2/15, 0)$

$\sum w_i x_i$

20

$\sum p_i x_i$

28.2

容量消耗过快

- 按物品重量的非降次序

$(0, 2/3, 1)$

20

31

效益值增长缓慢

- 按 p_i/w_i 比值的非增次序

$(0, 1, 1/2)$

20

31.5

每消耗单位容量，
将获得当前最大的
效益值





算法5.2 背包问题的贪心算法

先将物品按 p_i/w_i 比值的非增次序排序(降序)

```

procedure GREEDY-KNAPSACK(P,W,M,X,n)
real  P(1:n), W(1:n), X(1:n), M, cu; //X表示解向量
integer  i, n;
X ← 0
cu ← M //cu表示背包剩余容量
for i ← 1 to n do
    if (W(i)>cu) then exit endif
    X(i) ← 1
    cu ← cu-W(i)
repeat
if (i≤n) then X(i) ← cu/W(i) //物品i放入一部分
end GREEDY-KNAPSACK
    
```

思考1： 算法的时间复杂度？ $O(n)$

思考2： 算法中实现了约束条件，
如何确认当前可行解是最优解？

证明

最优量度标准证明的基本思想

- 把算法的贪心解与假定的最优解相比较，如果这两个解不同，就去找开始不同的第一个 x_i ，然后设法用贪心解的 x_i 去替换掉最优解中的 x_i ，然后证明最优解在分量替换前后的目标函数值无任何变化。
- 反复进行这种替换，直到新产生的最优解与贪心解完全一样，从而证明了贪心解就是最优解。



定理5.1 背包问题的贪心解最优证明

定理5.1: 如果 $p_1/w_1 \geq p_2/w_2 \geq \dots \geq p_n/w_n$, 则算法GREEDY-KNAPSACK对于给定的背包问题实例生成一个最优解。

证明: 设 $X=(x_1, \dots, x_n)$ 是算法所生成的贪心解。

1. 如果所有的 x_i 等于1, 显然这个解就是最优解。否则, 设 j 是使 $x_j \neq 1$ 的最小下标, 由算法可知:

对于 $1 \leq i < j$, $x_i = 1$;

对于 $j < i \leq n$, $x_i = 0$;

对于 $i=j$, $0 \leq x_j < 1$ 。

$x_1 \dots x_{j-1} \boxed{x_j} x_{j+1} \dots x_n$
 $1, \dots, 1, x_j, 0, \dots, 0$





2. 若 X 不是最优解, 则必存在一个最优解 $Y=(y_1, \dots, y_n)$, 使得 $\sum p_i y_i > \sum p_i x_i$ 。不失一般性, 假定 $\sum w_i y_i = M$, 设 k 是使得 $y_k \neq x_k$ 的最小下标, 显然, 这样的 k 必定存在。

$$\begin{array}{ccccccc} x_1 & \dots & x_{k-1} & x_k & x_{k+1} & \dots & x_n \\ \parallel & & \parallel & \neq & & & \\ y_1 & \dots & y_{k-1} & y_k & y_{k+1} & \dots & y_n \end{array}$$

$$\begin{array}{ccccccc} x_1 & \dots & x_{j-1} & x_j & x_{j+1} & \dots & x_n \\ 1, & \dots, & 1, & x_j, & 0, & \dots, & 0 \\ y_1 & \dots & y_{j-1} & y_j & y_{j+1} & \dots & y_n \end{array}$$

3. 可以推断 $y_k < x_k$ 成立, k 与 j 的关系有三种可能的情况:

① 若 $k < j$, 则 $x_k = 1$, 因 $y_k \neq x_k$, 从而 $y_k < x_k$;

② 若 $k = j$, 如果 $x_k < y_k$, 因为 $\sum w_i x_i = M$, 则有 $\sum w_i y_i > M$, 这与 $\sum w_i y_i = M$ 矛盾, 如果 $y_k = x_k$, 与假设 $y_k \neq x_k$ 矛盾。故 $k = j$ 时必有 $y_k < x_k$;

③ 若 $k > j$, 因为 $\sum w_i x_i = M$, 则有 $\sum w_i y_i > M$, 这是不可能发生的(与 $\sum w_i y_i = M$ 矛盾)。

故不存在 $k > j$ 的情况。

4.现在把 y_k 增加到 x_k , 那么必须从 $(y_{k+1}, \dots, y_n)$ 中减去同样多的量, 使得总容量仍为 M 。这构造一个新的解 $Z = (z_1, \dots, z_n)$, 其中 $z_i = x_i$, $1 \leq i \leq k$ 。并且

$$\sum_{k < i \leq n} w_i (y_i - z_i) = w_k (z_k - y_k)$$

$z_1 \dots z_{k-1}$	z_k	$z_{k+1} \dots z_n$
$\parallel$	$\parallel$	$\parallel$
$x_1 \dots x_{k-1}$	x_k	$x_{k+1} \dots x_n$
$\parallel$	$\parallel$	$\vee$
$y_1 \dots y_{k-1}$	y_k	$y_{k+1} \dots y_n$

5.因此, 对于 Z 有:

$$\begin{aligned} \sum_{1 \leq i \leq n} p_i z_i &= \sum_{1 \leq i \leq n} p_i y_i + (z_k - y_k) p_k - \sum_{k < i \leq n} (y_i - z_i) p_i \\ &= \sum_{1 \leq i \leq n} p_i y_i + (z_k - y_k) w_k p_k / w_k - \sum_{k < i \leq n} (y_i - z_i) w_i p_i / w_i \\ &\geq \sum_{1 \leq i \leq n} p_i y_i + [(z_k - y_k) w_k - \sum_{k < i \leq n} (y_i - z_i) w_i] p_k / w_k \\ &= \sum_{1 \leq i \leq n} p_i y_i \end{aligned}$$

元素按 p_i/w_i 非增次序排列。

即 $\sum p_i z_i \geq \sum p_i y_i$

6. 若 $\sum p_i z_i > \sum p_i y_i$ ，则 Y 不可能是最优解，所以 $\sum p_i z_i = \sum p_i y_i$ 。如果 $Z=X$ ，则 X 就是最优解；否则重复上面的讨论，每次 Y 中少一个和 X 不同的值，最终把 Y 转换成 X ，从而证明了 X 也是最优解，证毕。





5.3 带有期限的作业调度问题

- 问题描述
- 问题实例
- 贪心法实现思想
- 贪心解最优证明
- 可行调度证明
- 基于直接插入排序的贪心算法
- 优化思想
- 集合树(补充)
- 基于集合树的贪心算法



问题描述

- 假定只能在一台机器上处理 n 个作业，每个作业均可在单位时间内完成；又假定每个作业 i 都有一个截止期限 $d_i > 0$ (d_i 是整数)，当且仅当作业 i 在它的期限截止之前被完成时获得 $p_i > 0$ 的效益。
- 该问题的一个可行解
 - 是这 n 个作业的一个子集合 J ： J 中的每个作业都能在各自的截止期限之前完成，可行解的效益值是 J 中这些作业的效益之和 $\sum p_j$
- 该问题的一个最优解
 - 具有最大效益值的可行解

约束条件

目标函数

问题实例

• $n=4$

• $(p_1, p_2, p_3, p_4) = (100, 10, 15, 20)$

• $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$

可行解	处理顺序	效益值
1	1	100
2	2	10
3	3	15
4	4	20
1, 2	2, 1	110
1, 3	1, 3	115
1, 4	4, 1	120
2, 3	2, 3	25
3, 4	4, 3	35



算法5.3 贪心法实现思想

当前作业一旦满足约束条件，
问题就能获得的最大效益增量

- 基于目标函数 $\sum p_j$ 设计最优量度标准

- 将各作业按效益 p_i 降序排列: $p_1 \geq p_2 \geq \dots \geq p_n$

procedure GREEDY_JOB(D, J, n)

// 作业按 $p_1 \geq p_2 \geq \dots \geq p_n$ 的次序输入; 期限值 $D(1:n) \geq 1$; J是最优解//

J $\leftarrow$ {1}

for i $\leftarrow$ 2 to n do

 if (J $\cup$ {i}的所有作业都能在它们的截止期限前完成)

 then J $\leftarrow$ J $\cup$ {i}

 endif

repeat

end GREEDY_JOB

思考1: 算法GREEDY_JOB是否能提供一个最优解?

思考2: 对于给定的作业集合J, 如何确定它是可行解?

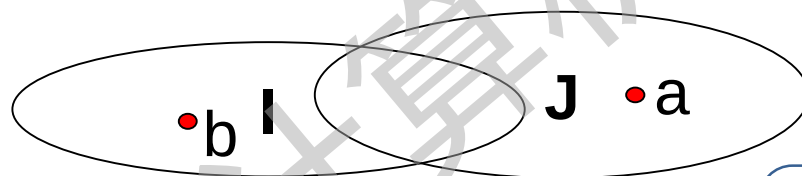
定理5.2 作业调度问题的贪心解最优证明

思考1得证

定理5.2：算法GREEDY_JOB所描述的贪心方法总是得到一个最优解。

• 证明思路：

- J是贪心方法求出的作业集合，I是一个最优解的作业集合。可以证明J和I具有相同的效益值，从而J也是最优解。
- 证明当 $I \neq J$ 时：



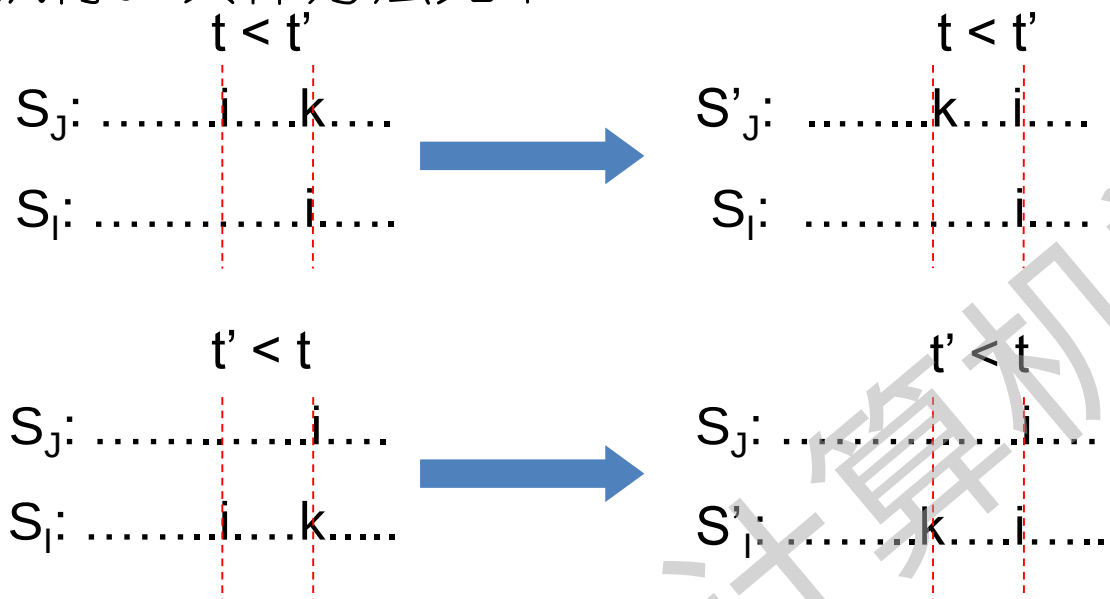
由贪心方法可知，对于在I中又不在J中的所有作业b，都有 $p_a \geq p_b$ 。这是因为，若 $p_b > p_a$ ，则贪心方法会先于a考虑b，并把b计入到J中去。

a是使得 $a \in J$ 且 $a \notin I$ 的一个最高效益值的作业

思考：J和I的元素个数相同么？



- 设 S_J 和 S_I 分别是 J 和 I 的可行调度表。令调度表中相同作业在相同的时间片执行。具体方法如下：



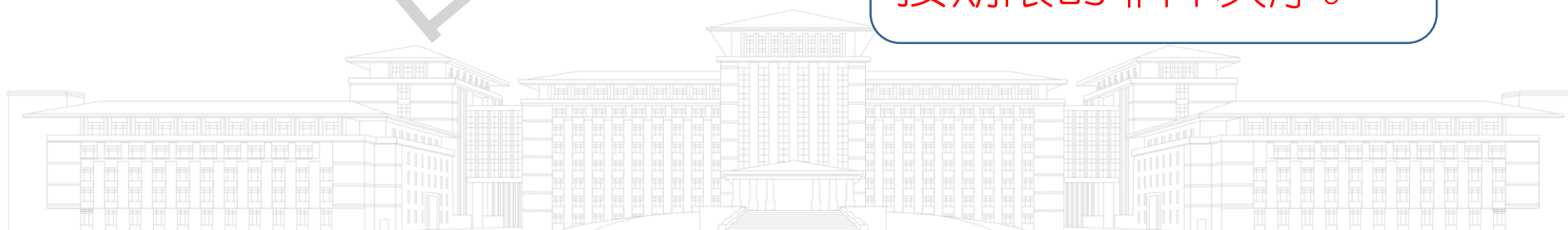
- 用 J 中作业 a 替换掉 I 中同时间片调用的作业 b ，得到作业集合 $I' = I - \{b\} \cup \{a\}$ 的一个可行调度表。显然， I' 的效益值不小于 I 的效益值，并且 I' 比 I 少一个与 J 不同的作业。
- 重复应用上述转换，使 I 在不减效益值的情况下转换成 J ，因此 J 也必定是最优解，证毕。



如何判断J是可行解的策略？

- 检验J的所有可能的排列：
 - $\sigma = i_1, i_2, \dots, i_k$ 是J中作业的一种排列；
 - 完成作业 i_j 的最早时间是 j , $1 \leq j \leq k$;
 - 若排列中的每个作业的 $d_{i_j} \geq j$, 则 σ 是一个允许的调度序列, J 是一个可行解; 否则, 检验其他排列形式。

检验一种特殊的排列：
按期限的非降次序。



定理5.3 可行调度证明

思考2得证

计算机科学与技术学院



定理5.3：设 J 是 k 个作业的集合， $\sigma=i_1, i_2, \dots, i_k$ 是 J 中作业的一种排列，它使得 $d_{i_1} \leq d_{i_2} \leq \dots \leq d_{i_k}$ 。 J 是一个可行解，当且仅当 J 中的作业可以按照 σ 的次序而又不违反任何一个期限的情况来处理。

• 证明思路：

← • 如果 J 中的作业可以按照 σ 的次序而又不违反任何一个期限，则 J 是一个可行解。

→ • 若 J 是可行解，则必存在 $\sigma'=r_1, r_2, \dots, r_k$ ，使得 $d_{r_j} \geq j$ ， $1 \leq j \leq k$ 。

• 假设 $\sigma' \neq \sigma$ ，令 a 是使得 $r_a \neq i_a$ 的最小下标；设 $r_b = i_a$ ，显然 $b > a$ 。

• 在 σ' 中交换 r_a 与 r_b 的位置，产生新的可行排列 σ'' 。

• 连续使用这种方法，就将 σ' 转换成 σ 且不违反任何一个期限。

$$d_{r_b} = d_{i_a} \leq d_{r_a}$$

$\sigma' : \dots r_a \dots r_b \dots$

$\sigma'' : \dots r_b \dots r_a \dots$

$\sigma : \dots i_a \dots$

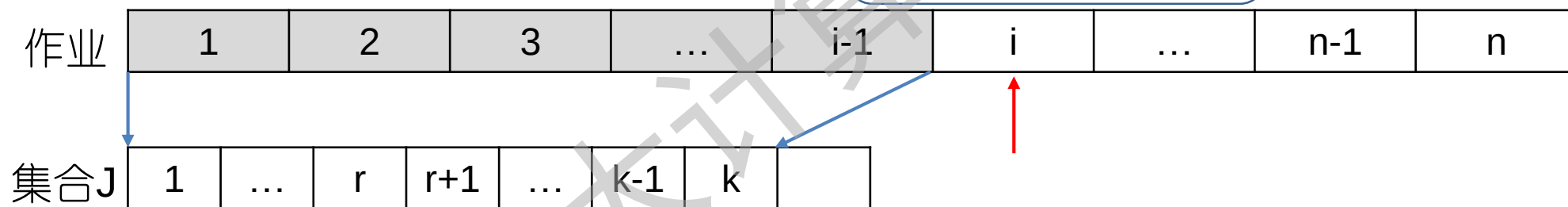
$\sigma : \dots i_a \dots$

思考： r_a 延后执行是否可行？

基于定理5.3检验可行解

- 假设已处理了 $i-1$ 个作业，有 k 个作业已存入 J 中，且 $D[J(1)] \leq D[J(2)] \leq \dots \leq D[J(k)]$
- 在 J 中从后向前为作业 i 寻找位置 $r+1$ ， i 插入 $r+1$ 位置后， J 中作业仍按照期限值从小到大排列，且不违反期限值

用直接插入法比较期限值大小



$r+1 \leq l \leq k, D[J(l)] > l$ 时，允许向后移动1位
 $D[J(r)] \leq D[i], D[i] > r, D[J(l)] > D[i]$

思考：为什么 $D[J(l)] > l$ 时，才允许向后移动1位？

算法5.4 基于直接插入排序的贪心算法



```
procedure JS(D, J, n, k)
integer D(0:n), J(0:n), i, k, n, r
D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
  r ← k
  while D(J(r)) > D(i) and D(J(r)) ≠ r do
    r ← r - 1
  repeat
    if D(J(r)) ≤ D(i) and D(i) > r then
      for l ← k to r+1 by -1 do
        J(l+1) ← J(l)
      repeat
        J(r+1) ← i; k ← k+1;
      until false
    endif
  repeat
end JS
```

寻找r+1位置

位置r+1到k的作业依次后移一位

作业i插入到r+1位置

- 两个关键参数:
 - 作业数n
 - J的最终作业数 $s \leq n$
- 内层while至多循环 $k \leq s \leq n$ 次
- 外层for执行循环n-1次
- JS算法的时间复杂度为 $O(n^2)$

思考1: 算法JS的时间复杂度? $O(n^2)$ 思考2: 能否降低时间复杂度?

- 若还没给作业*i*分配处理时间, 则分给它时间片 $[\alpha-1, \alpha]$, 其中 α 是不大于 d_i 的最大空时间片。若不存在这样的 α , 则*i*不计入*J*中。
- 旨在去掉内层while循环, 算法的复杂度由 $O(n^2)$ 降低到接近于 $O(n)$

<i>i</i>	1	2	3	4	5
p_i	20	15	10	5	1
d_i	2	2	1	3	3

一维数组*J*

①. 计入作业1

②. 计入作业2

③. 舍弃作业3

④. 计入作业4

⑤. 舍弃作业5

1	2	3
	1	
2	1	
2	1	
2	1	4
2	1	4

思考: 如何找到 α ?

用空间换时间

定义一个数组记录期限*d*当前可用的最大空时间片?

一维数组F



- 定义一维数组F， $F(d_j)$ 表示时间期限为 d_j 的作业j当前可用的最大可用空时间片。

作业i	1	2	3	4	5	F初始	1	2	3	4
d_i	3	3	4	3	4	作业1	1	2	2	4
						作业2	1	2	1	4
						作业3	1	1	1	3
						作业4	0	0	0	0
						作业2	1	1	1	4
						作业3	1	1	1	1

无需维护J，但
需要维护F，还
需要顺序遍历

为提高效率，不再对所有的F(i)维护





一维数组P

- 定义一维数组P模拟集合树，同一棵树中的结点具有相同的最大可用时间片，即根结点j对应的F(j)。
 - 若 $P(i)=-k$,则i是根结点， $k>0$ 表示树中结点个数。
 - 若 $P(i)=k$,则i是分支/叶结点， $k>0$ 是i的父结点。



优化总结：考查作业i时，根据d(i)的值先访问数组P，再访问数组F，如果有可用空时间片，则i加入到J中。

集合树(补充)

- 集合树表示一个集合
 - 树中每个结点只设置PARENT信息段。
 - 对于非根结点 i , $PARENT(i)$ 存放着 i 的父结点下标。
 - 对于根结点 i , $PARENT(i) = -k$, $k > 0$ 表示树中结点个数。
- 集合树的操作
 - 查找: $FIND(i)$ 表示基于压缩规则查找结点 i 的根结点。
 - 合并: $UNION(i, j)$ 表示基于加权规则合并两个集合树 i 和 j 。



集合树的查找与合并操作

• **FIND(i)**:

压缩规则

- 找到结点 i 所在树的树根 j ，并且将从结点 i 到树根 j 的路径上所有祖先结点 k 的 $\text{PARENT}(k)$ 都变为 j ，并返回根结点 j

• **UNION(i, j)**:

- 将以 i 和 j 为根的两棵树合并为一棵树，根据**加权规则**，如果树 i 的结点数少于树 j 的结点数，则 j 为 i 的父亲，反之， i 为 j 的父亲



算法5.5设计思想

- 变量定义

- $b = \min\{n, \max\{d_j\}\}$, 可能分配的时间片范围
- 数组 P , 模拟集合树, 结点 i 对应期限值 i , 若 $P(i) < 0$, 则 i 是根结点, 否则, 是结点 i 的父结点下标
- 数组 F , $F(i)$ 表示期限值为 i 的作业当前可用的最大时间片, i 在 P 中是根结点时 ($P(i) < 0$), 该数据有效



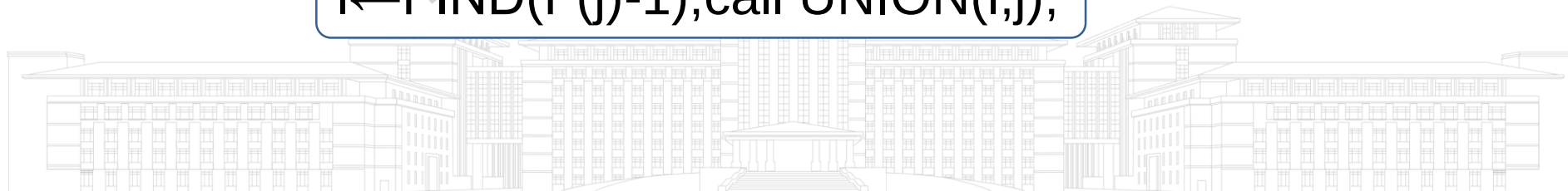
算法5.5设计思想

- 初始
 - $p_1 \geq p_2 \dots \geq p_n$
 - $b = \min\{n, \max\{d_j\}\}$
 - $F(i) \leftarrow i$
 - $P(i) \leftarrow -1$
- 依次检验每个作业 w
 - 寻找 $D(w)$ 所在树的根结点 j

$j \leftarrow \text{FIND}(\min(n, D(w)))$
 - 如果 $F(j) \neq 0$, $F(j)$ 时间片可以分配给作业 w , 因此, 将 w 并入到解集合 J 中,

$k \leftarrow k+1; J(k) \leftarrow w;$
 - 寻找 $F(j)-1$ 所在树的根结点 l , 将 l 和 j 所在的两个树合并到一起。
 - $F(j) \leftarrow F(l)$

$l \leftarrow \text{FIND}(F(j)-1); \text{call UNION}(l, j);$
- 使用加权规则的合并算法 **UNION(i, j)**
- 使用压缩规则的查找算法 **FIND(i)**



算法5.5 基于集合树的贪心算法

```

procedure FJS(D,n,b,J,k)
// 找最优解 $J=J(1),\dots,J(k)$ , 已知 $p_1 \geq p_2 \dots \geq p_n$ ,  $b=\min\{n,\max\{d_j\}\}$ 
integer b, D(n), J(n), F(0:b), P(0:b)
for i  $\leftarrow$  0 to b do
    F(i)  $\leftarrow$  i; P(i)  $\leftarrow$  -1
repeat
k $\leftarrow$  0
for i  $\leftarrow$  1 to n do
    j $\leftarrow$  FIND(min(n, D(i)))
    if F(j)  $\neq$  0 then k  $\leftarrow$  k+1; J(k)  $\leftarrow$  i;
        l $\leftarrow$  FIND(F(j)-1); call UNION(l,j);
        F(j)  $\leftarrow$  F(l)
    endif
repeat
End FJS
    
```

接近 $O(n)$

思考：为什么集合树能将算法的复杂度降到接近 $O(n)$?

算法示例



- 设 $n=7$
- $(p_1, p_2, \dots, p_7) = (35, 30, 25, 20, 15, 10, 5)$
- $(d_1, d_2, \dots, d_7) = (4, 2, 4, 3, 4, 8, 3)$

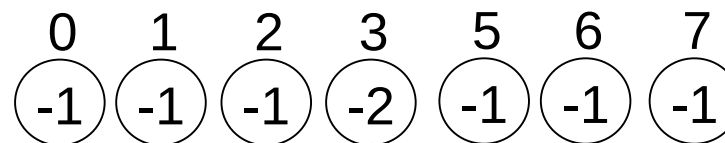
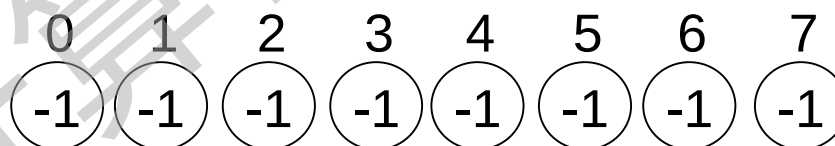
考虑
作业

F 0 1 2 3 4 5 6 7

无 0 1 2 3 4 5 6 7

1 0 1 2 3 **3** 5 6 7

树P



J

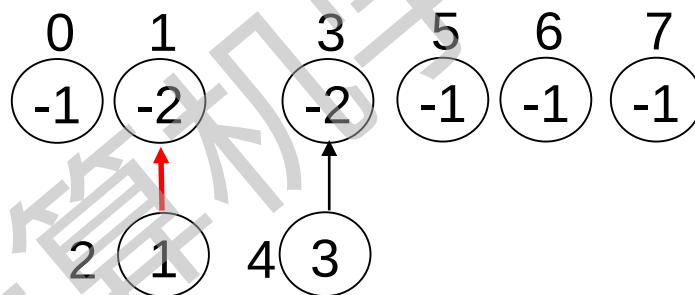
$\emptyset$

{1}

- 设 $n=7$
- $(p_1, p_2, \dots, p_7) = (35, 30, 25, 20, 15, 10, 5)$
- $(d_1, d_2, \dots, d_7) = (4, 2, 4, 3, 4, 8, 3)$

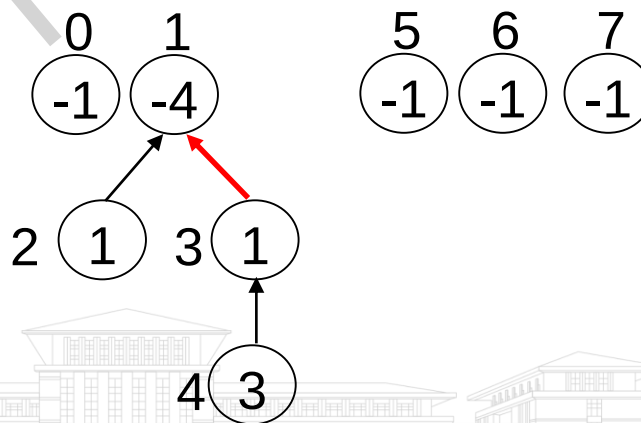
考虑
作业

F	0	1	2	3	4	5	6	7
2	0	1	1	3	3	5	6	7



{1,2}

3	0	1	1	1	3	5	6	7
---	---	---	---	---	---	---	---	---



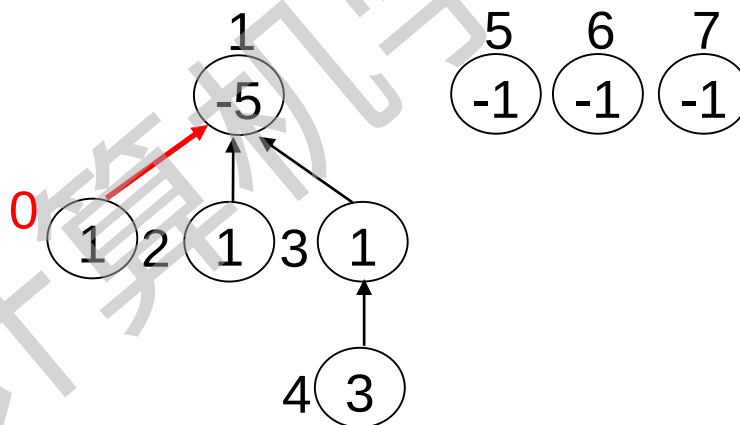
{1,2,3}

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7) = (35, 30, 25, 20, 15, 10, 5)$
- $(d_1, d_2, \dots, d_7) = (4, 2, 4, 3, 4, 8, 3)$

考虑
作业

F 0 1 2 3 4 5 6 7

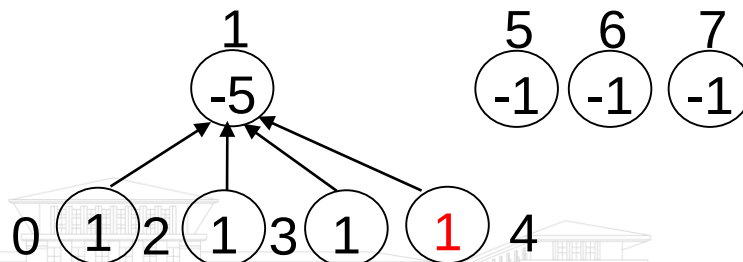
4 0 0 1 1 3 5 6 7



{1, 2, 3, 4}

5
(舍弃)

0 0 1 1 3 5 6 7



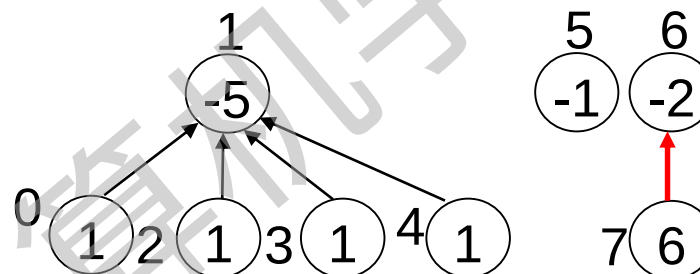
{1, 2, 3, 4}

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7)=(35,30,25,20,15,10,5)$
- $(d_1, d_2, \dots, d_7)=(4,2,4,3,4,8,3)$

考虑
作业

F 0 1 2 3 4 5 6 7

6 0 0 1 1 3 5 6 **6**



$\{1,2,3,4,6\}$

7
(舍弃)

0 0 1 1 3 5 6 6

同上

最优解 $J = \{1,2,3,4,6\}$, 处理次序 4,2,3,1,6, 效益值 120



5.4 最小生成树问题

- 问题回顾
- **Kruskal**算法描述
- 实例运行
- 贪心策略分析
- 贪心解最优证明

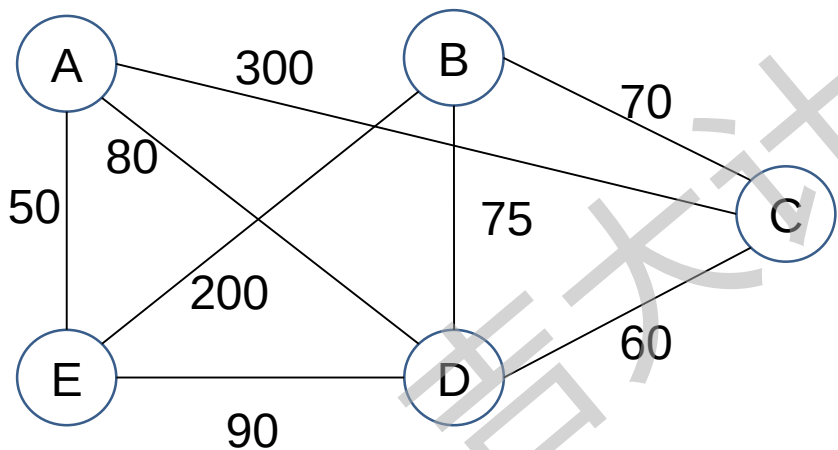
贪心法解决的一个有名问题是最小生成树问题，本节介绍最小生成树的Kruskal方法



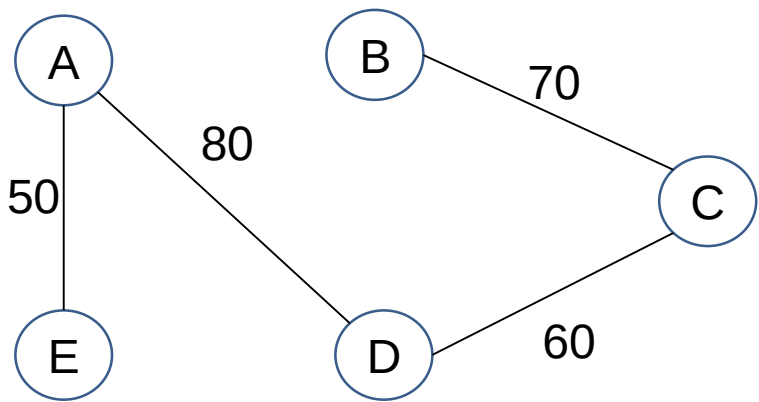
问题回顾

- 最小生成树的定义：

- 设 $G=(V,E)$ 是一个加权无向连通图。 V 表示顶点集合， E 表示边集合。 G 的一棵生成树是一棵无向树 $T=(V, E')$ ，其中 E' 是 E 的子集。生成树的权是 E' 的所有权之和。 G 的最小生成树是 G 的具有最小权值的生成树。



加权连通无向图



最小生成树

Kruskal算法描述

最坏情况下 $O(n^2 \log n)$

- input: 一个加权连通无向图 $G=(V,E)$

- output: 图 G 的一棵最小生成树

$T \leftarrow \emptyset$ //存储选中的边

while T 所含的边数少于 $n-1$ do

 从 E 中选择一条最小权值的边 (v,w) ; 从 E 中删除该边

 if (将 (v,w) 加入 T 中没有形成一个回路)

 then 将 (v,w) 加入 T 中

 else 放弃 (v,w)

 endif

repeat

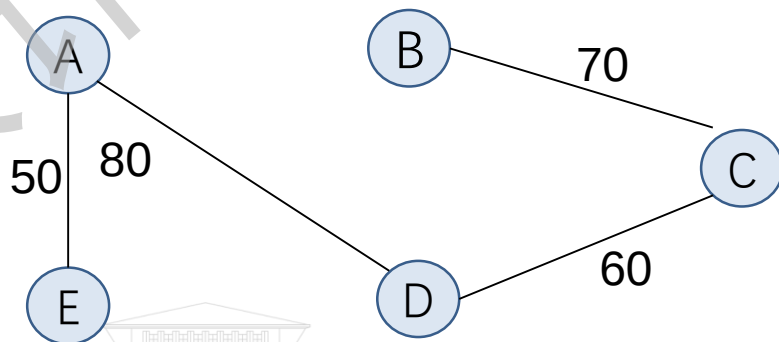
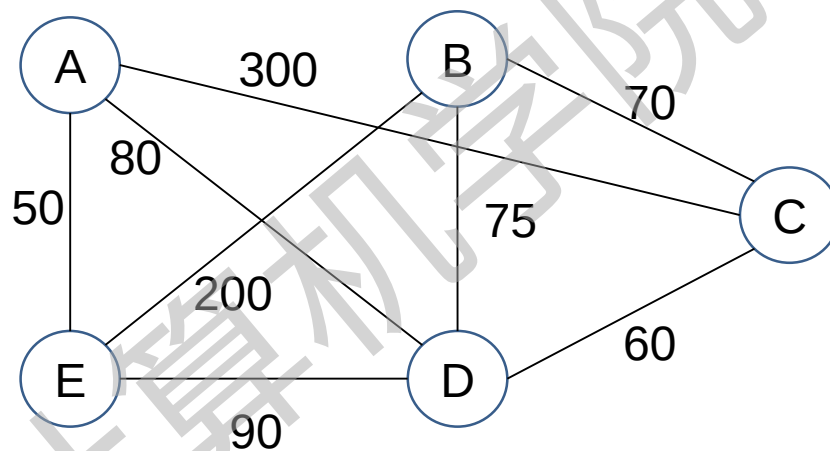
n 表示顶点的个数。从边的个数 e 入手分析，边排序时间复杂度为 $O(e \log e)$ 。考虑连通无向图边的个数最多为 $n(n-1)/2$, 结论得证。



实例运行

• 边的权值排序:

- 边(AE) 50
- 边(CD) 60
- 边(BC) 70
- 边(BD) 75
- 边(AD) 80
- 边(ED) 90
- 边(BE) 200
- 边(AC) 300



贪心策略分析

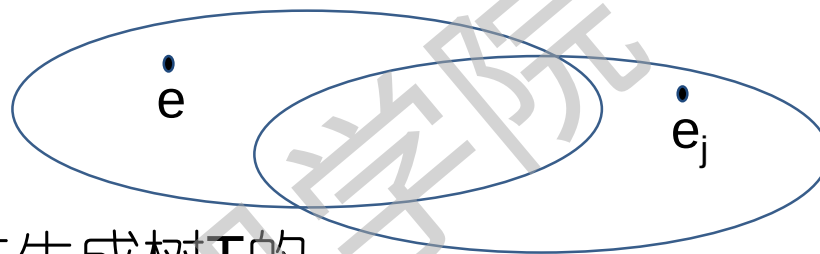
- 问题原型
 - 构造加权无向连通图 $G=(V,E)$ 的最小生成树 $T=(V, E')$, $|V|=n$
- 约束条件:
 - T 中 $|V|=n$; T 中 $|E'|= n-1$, $E' \subseteq E$; T 中没有环
- 目标函数:
 - 权值和最小
- 量度标准:
 - 权值从小到大排列

把目标函数作为度量标准

思考: 该量度标准是否是最优量度标准?



证明：Kruskal算法对于每一个无向连通图 G 产生一棵最小生成树。



贪心法生成树 T 的
边集合 $E(T)$

最小生成树 T' 的
边集合 $E(T')$

- 证明路线：
 - 用 e 替换掉 e_j ，获得新的可行解 T'' ，证明新解也是最小生成树。反复替换，从而命题得证。
- 证明难点：
 - 怎样选择 e 和 e_j ？
 - 怎样确定 e 和 e_j 的成本关系？

根据证明路线要求，希望 $e_j \geq e$ 。因此，要反向证明 $e_j < e$ 是不合理的。

证明：

1. 设 T 是Kruskal算法产生的 G 的生成树， T' 是 G 的最小成本生成树。现在要证明 T 和 T' 具有相同的成本。
2. 设 $E(T)$ 和 $E(T')$ 分别是 T 和 T' 的边集。若 n 是 G 中的结点数，则 T 和 T' 都有 $n-1$ 条边。
3. 若 $E(T)=E(T')$ ，则 T 显然就是最小生成树。
4. 若 $E(T) \neq E(T')$ ，则假设 e 是一条使得 $e \in E(T)$ 但 $e \notin E(T')$ 的最小成本的边。
5. 把 e 加入到 T' 中，则一定会产生一个环。假设 $e, e_1, e_2, \dots, e_k$ 是这个环。
6. 可知 $e_1, e_2, \dots, e_k$ 中至少有一条不在 $E(T)$ 中，如若不然，则 T 也包含这个环，与算法矛盾。设 e_j 是这个环中使得 $e_j \notin E(T)$ 的一条边。

4-6：选择 e 和 e_j



7. 如果 e_j 比 e 有更小的成本，则Kruskal算法会在 e 之前考虑 e_j ，并把 e_j 计入 T 中。与假设矛盾，因此 e_j 不比 e 有更小的成本，记作 $c(e_j) \geq c(e)$ 。
8. 重新考虑 $E(T') \cup \{e\}$ 的图。删去边 e_j ，产生新树 T'' ，由于 $c(e_j) \geq c(e)$ ，因此 T'' 的成本不比 T' 大。因此 T'' 也是一棵最小生成树。
9. 反复上述转换，树 T' 转换成 T 而在成本上没有任何增加，故 T 是一棵最小生成树，证毕。



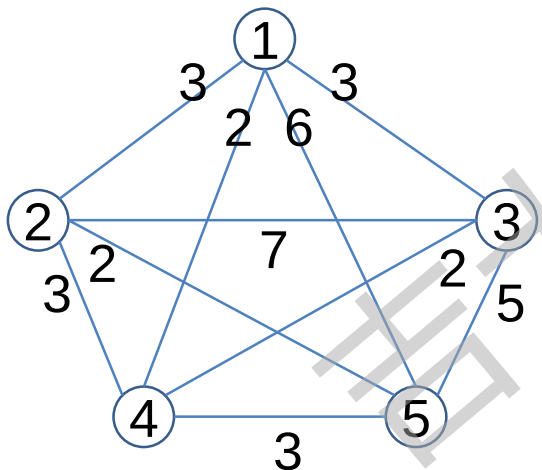
5.5 货郎担问题

- 货郎担问题也叫旅行商问题，即TSP问题 (Traveling Salesman Problem)，是数学领域中著名问题之一。
- **问题描述：**某售货员要到若干个城市销售货物，已知各城市之间的距离，要求售货员从某一城市出发并选择行程路线，使每个城市经过一次，最后回到原出发城市，而总路程最短。



贪心策略

- 根据目标函数，选择距离当前城市最短距离的城市
 - 从某一个城市开始，每次选择一个新城市，直到所有的城市被走完。
 - 在选择下一个城市时，只考虑当前情况，保证当前选择的城市距离最小且不形成回路。



	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞

贪心策略只能求出问题的近似解。

从城市1出发

	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞

1->4->3->5->2->1, 总距离14

从城市2出发

	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞

2->5->4->1->3->2, 总距离17

思考：贪心策略求出n个点出发的回路，是否能找到最优解？时间复杂度是多少？

5.6 小结

- 贪心法特点：
 - 多步判断，只顾眼前
 - 不考虑子问题的计算结果的优劣
 - 只考虑子问题当前决策的优劣
 - 有时只能获得局部最优解
 - 全局最优解需要经过正确性证明
- 对于许多NP难的组合优化问题，近似算法是一种比较可行的途径
 - 贪心法常用于这些近似算法的设计中。



5.6 小结

- 5.1 一般方法
 - 掌握约束条件、可行解、最优解、目标函数的定义；理解量度标准的意义；掌握贪心方法适用的问题特点和求解思想等基本知识。能掌握贪心法求解问题的一般过程。
- 5.2 背包问题
- 5.3 带有期限的作业调度问题
 - 掌握贪心法最优解证明的一般方法和贪心算法设计的一般思想。深入理解贪心法时间复杂度的影响因素，掌握贪心法优化思路。



5.6 小结

●5.4 最小生成树问题

- 深入理解贪心法适用的问题特征，掌握复杂问题求解办法。掌握贪心法求解复杂问题时最优解的证明方法。

●5.5 货郎担问题

- 理解贪心法近似解和实际最优解间的差异。

能够识别出适合贪心法的可计算性问题、独立设计算法和分析算法复杂度。





本章结束

